

WSCAD Code Challenge

TAKE-HOME ASSIGNMENT: AGENTIC AI SOLUTION

Goal

Design and implement a small but realistic Agentic AI system that demonstrates your understanding of:

- Agentic workflows
- Applied LLM patterns (RAG, reasoning, decision-making, tool usage)
- Production-oriented engineering thinking

You may use ChatGPT, Copilot, or other code-generation tools.

Problem Statement

Build an Agentic AI system that processes technical support tickets and:

- Understands the problem
- Retrieves relevant knowledge from a local knowledge base (RAG)
- Decides on next actions
- Produces a structured and explainable result

Provided Input

Knowledge Base

You will receive a folder containing short documents (Markdown / TXT):

- Licensing and activation
- Installation requirements
- Common errors and troubleshooting

These documents are the only authoritative knowledge source.

Example Support Tickets

You will receive a JSON file containing example tickets. Each ticket includes:

- ticket_id
- Free-text description
- Optional / incomplete metadata (product, version, OS, urgency)

Metadata may be missing or unreliable by design.



Sample Outputs

Sample outputs are provided to illustrate:

- The expected response structures
- The level of detail
- Both resolvable cases and clarification-required cases

The sample outputs are for guidance only and must not be treated as ground truth or hard-coded logic.

What You Must Design

You are expected to design and justify the following components:

1. Agent Architecture

- How many agents exist
- What each agent is responsible for
- How agents exchange information

2. Classification Logic

Define:

- Issue categories (e.g., licensing, installation, runtime error)
- Priority levels (e.g., low / medium / high)

Explain how classification decisions are made.

3. RAG Strategy

Describe:

- Chunking approach
- Retrieval strategy
- How retrieved context is used in reasoning
- How hallucination is mitigated

4. Clarification Logic

Define:

- How missing critical information is detected
- How follow-up questions are generated



- When the system proceeds vs. when it stops

5. Output Structure

Your system's output must include at minimum:

- Proposed solution or recommendation
- Classification (category + priority)
- Confidence level
- Short reasoning trace
- Optional follow-up questions

Optional Enhancements

- Use of agent frameworks (e.g., Semantic Kernel, LangChain, LangGraph)
- Additional test tickets
- Simple evaluation or scoring logic

Not Required

- UI or frontend
- Model fine-tuning
- External data sources
- Perfect accuracy

Evaluation Criteria

We will evaluate:

- Quality of agentic design (not prompt chaining)
- Correct use of RAG
- Reasoning transparency
- Engineering clarity and structure
- Production mindset and trade-off awareness

